Ministry of Higher Education and Scientific Research

Ziane Achour University of Djelfa
Faculty of Science and Technology
Department of Computer Science

BadgeAppFTP

Employee Badge Management System

Technical Project Report

Author: BELHADJ Mohamed Ali

Status: PhD Student in Computer Science

Institution: Ziane Achour University – Djelfa, Algeria

Group: 4

GitHub: github.com/blhjmedali/BadgeAppFTP

April 2026

Contents

1 Introduction **4**
- 1.1 Project Overview . . . 4
- 1.2 Motivation . . . 4
- 1.3 Project Scope . . . 5
- 1.4 Document Structure . . . 5

2 Technical Background **6**
- 2.1 Python and Django Framework . . . 6
- 2.2 Database: SQLite . . . 6
- 2.3 PDF Generation . . . 7
- 2.4 QR Code Integration . . . 7
- 2.5 File Storage . . . 7
- 2.6 Project Dependencies . . . 7

3 System Architecture **8**
- 3.1 Overall Architecture . . . 8
- 3.2 Django Application Structure . . . 8
- 3.3 File System Layout . . . 8
- 3.4 Authentication and Security . . . 9

4 Functional Features **10**
- 4.1 Authentication Module . . . 10
- 4.2 Dashboard . . . 10
- 4.3 Employee Management . . . 11
  - 4.3.1 Adding a New Employee . . . 11
  - 4.3.2 Employee ID Format . . . 11
  - 4.3.3 Bulk CSV Upload . . . 11
- 4.4 Badge Generation and Preview . . . 11
  - 4.4.1 Badge Preview . . . 12
  - 4.4.2 PDF Download . . . 12
- 4.5 Navigation and Sidebar . . . 12

5 Implementation Highlights **13**
- 5.1 Employee Model . . . 13
- 5.2 Badge PDF Generation View . . . 14

5.3 Deployment Script 15

6 Application Screenshots **16**

6.1 Login Page 16

6.2 Dashboard 17

6.3 Add New Employee Form 17

6.4 Badge Preview and PDF Export 18

6.5 Project File Structure (Terminal) 18

7 Testing and Validation **19**

7.1 Manual Testing 19

7.2 Observed Results 19

8 Conclusion and Future Work **21**

8.1 Conclusion 21

8.2 Future Work 21

List of Figures

3.1 High-level system architecture of BadgeAppFTP 8

6.1 Login page – Administrator authentication interface 16
6.2 Main dashboard – System overview with metric cards and recent employees 17
6.3 Add New Employee form – Data entry with photo upload 17
6.4 Badge Preview page – Rendered identity card for BELHADJ Mohamed Ali 18
6.5 PowerShell terminal showing project directory structure and launch command 18

Chapter 1

Introduction

1.1 Project Overview

BadgeAppFTP is a web-based **Employee Badge Management System** designed to automate and streamline the process of generating, managing, and distributing corporate identity cards. Built using the Django web framework and Python, the application provides a complete pipeline from employee registration to printable PDF badge generation.

The system targets organizations that require a fast, reliable, and professional approach to issuing employee identification. By consolidating employee data entry, photo management, badge design, and PDF export into a single application, BadgeAppFTP significantly reduces the administrative burden associated with traditional badge creation workflows.

Project at a Glance

- **Application Name:** BadgeAppFTP (BadgeGen ID Management System)
- **Technology Stack:** Python, Django, SQLite, HTML/CSS, JavaScript
- **Deployment:** Local server (127.0.0.1:8000), extensible to production
- **Key Output:** Downloadable PDF employee badges with QR codes
- **Source Code:** github.com/blhjmedali/BadgeAppFTP

1.2 Motivation

In many institutions and companies, the process of generating employee ID cards is manual, time-consuming, and error-prone. Typically, human resources departments rely on graphic design tools such as Adobe Photoshop or Microsoft Word templates, which require significant effort for each individual card. Updating records, replacing photos, or changing department information introduces additional friction.

BadgeAppFTP was conceived to address these inefficiencies by offering:

- A centralized database of employee records with photo storage.

- Automated unique employee ID generation in a readable format (e.g., `EMP-2026-B6EE`).
- Instant badge preview and PDF export with embedded QR codes.
- Bulk CSV upload for large-scale onboarding scenarios.
- A clean, modern administrative interface accessible via any web browser.

1.3 Project Scope

The scope of this project encompasses the full stack of a web application:

1. **Backend:** Django models, views, URL routing, file handling, and PDF generation logic.
2. **Frontend:** Responsive HTML templates rendered server-side, styled with modern CSS.
3. **Database:** SQLite for development; the architecture supports migration to PostgreSQL.
4. **File System:** Management of employee photo uploads and generated PDF badges.
5. **Security:** Session-based authentication protecting all administrative routes.

1.4 Document Structure

The remainder of this report is organized as follows. Chapter 2 presents the technical background and the technology stack selected. Chapter 3 describes the system architecture. Chapter 4 details the functional features of the application. Chapter 5 discusses implementation highlights. Chapter 6 presents the application screenshots. Chapter 8 concludes the report and outlines future work.

Chapter 2

Technical Background

2.1 Python and Django Framework

The core of BadgeAppFTP is built on **Python 3** and the **Django** web framework. Django follows the Model-View-Template (MVT) architectural pattern, which cleanly separates data management, business logic, and presentation layers.

Django was chosen for this project due to:

- Its built-in ORM (Object-Relational Mapper), which abstracts database operations.
- The mature ecosystem of third-party libraries for PDF generation and image processing.
- A robust authentication system suitable for protecting administrative interfaces.
- The `manage.py` command-line tool, which simplifies migration and server management.

2.2 Database: SQLite

SQLite was selected as the database engine for the development and demonstration version of the application. It is serverless, requires no configuration, and stores the entire database in a single file (`db.sqlite3`), making it ideal for local deployment and prototyping.

The schema includes tables for:

- Employee records (name, job title, department, gender, photo path, employee ID).
- User accounts for authentication (Django's built-in `auth` system).

2.3 PDF Generation

Badge generation relies on a server-side PDF rendering library. The system produces pixel-perfect badges incorporating:

- Employee photo (circular crop style).
- Full name, job title, department, and unique employee ID.
- A QR code encoding the employee's ID for machine-readable verification.
- The BadgeGen corporate branding header.

2.4 QR Code Integration

Each generated badge contains a dynamically created **QR code** that encodes the employee's unique identifier. This enables rapid scanning and verification of identity cards using standard smartphone applications or dedicated QR readers, adding a layer of digital verifiability to the physical badge.

2.5 File Storage

Employee photos and generated badge PDFs are stored on the local file system under organized directories:

- `media/` – employee-uploaded photographs.
- `badges/` – generated PDF badge files.
- `static/` – CSS, JavaScript, and static assets.

2.6 Project Dependencies

Table 2.1 summarizes the primary project dependencies as declared in `requirements.txt`.

Table 2.1: Key project dependencies

Package	Purpose	Category
Django	Web framework (MVT, ORM, Auth)	Backend
Pillow	Image processing and photo handling	Media
ReportLab / WeasyPrint	PDF generation and badge rendering	Export
qrcode	QR code generation	Badge
SQLite3	Embedded relational database	Storage

Chapter 3

System Architecture

3.1 Overall Architecture

BadgeAppFTP follows a standard **server-side rendered web application** architecture. All business logic and data processing occur on the Django backend; the browser receives fully rendered HTML pages. Figure 3.1 illustrates the high-level architecture.

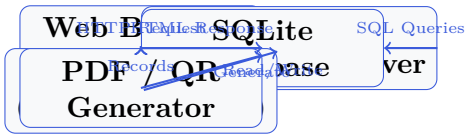


Figure 3.1: High-level system architecture of BadgeAppFTP

3.2 Django Application Structure

The project is organized into dedicated Django apps, each responsible for a specific domain:

- **`badgegen/`** – Core application containing models, views, and URL configuration for employee management and badge generation.
- **`employees/`** – Employee-specific logic: CRUD operations, photo handling, bulk CSV upload.
- **`badges/`** – Badge preview, PDF generation, and download endpoints.
- **`templates/`** – HTML templates for all pages (login, dashboard, employee form, badge preview).
- **`static/`** – CSS stylesheets and JavaScript files.
- **`media/`** – Runtime storage for uploaded employee photos.

3.3 File System Layout

The directory structure on disk (as observed during deployment) is shown below:

```
App2/
          .venv/              # Python virtual environment
          badgegen/           # Core Django app
          badges/             # Generated badge PDFs
          employees/          # Employee Django app
          media/              # Uploaded employee photos
          static/             # CSS, JS, images
          templates/          # HTML templates
          db.sqlite3          # SQLite database
          manage.py           # Django management script
          requirements.txt    # Python dependencies
          run-app.ps1         # PowerShell launch script
```

Listing 3.1: Project directory structure

3.4 Authentication and Security

All administrative routes are protected by Django's session-based authentication. An unauthenticated request to any management page redirects the user to the login view. The default administrator account (`admin`) is created during the initial setup via Django's `createsuperuser` command.

Chapter 4

Functional Features

4.1 Authentication Module

The application presents a login page (see Figure 6.1) at the root URL. Users must supply a valid **username** and **password** to gain access to the management interface. The login form includes:

- A username input field.
- A password input field with a show/hide toggle.
- A gradient Login button.
- Branded header with the BadgeGen logo.

Failed login attempts display an error message without revealing whether the username or password was incorrect, consistent with security best practices.

4.2 Dashboard

Upon successful login, the administrator is directed to the **Dashboard** (Figure 6.2). The dashboard provides a real-time overview of the system state through three metric cards:

Table 4.1: Dashboard metric cards

Card	Description	Data Source
Total Employees	Total registered employee count	`Employee.objects.count()`
Active Staff	Employees with active status	Filtered queryset
Departments	Number of unique departments	Distinct department query

The dashboard also displays a **Recently Added** table listing the latest employees with their names, unique IDs, departments, and a direct link to generate their badge. A **Quick Actions** sidebar provides one-click access to adding new employees, bulk CSV upload, and system settings.

4.3 Employee Management

4.3.1 Adding a New Employee

The **Add New Employee** form (Figure 6.3) collects the following fields:

- **Full Name** – The employee's complete name.
- **Job Title** – The employee's role or position.
- **Department** – A drop-down selector (Engineering, HR, Finance, Marketing, etc.).
- **Gender** – A drop-down selector (Male / Female).
- **Employee Photo** – A file upload input supporting common image formats (JPEG, PNG).

Upon submission, the system:

1. Validates the input fields.
2. Generates a unique employee ID in the format `EMP-YYYY-XXXX`.
3. Saves the employee record to the database.
4. Stores the uploaded photo in the `media/` directory.
5. Redirects to the employee list or dashboard.

4.3.2 Employee ID Format

Each employee receives an automatically assigned identifier following the pattern:

```
EMP-{YEAR}-{4-character hex suffix}
```

For example: `EMP-2026-B6EE` or `EMP-2026-0B07`. The hex suffix is derived from a UUID or random generation, ensuring uniqueness across all records.

4.3.3 Bulk CSV Upload

The **Bulk Upload** feature allows administrators to import multiple employees at once by uploading a CSV file. This is particularly valuable during organizational onboarding events where dozens or hundreds of employees need to be registered simultaneously.

4.4 Badge Generation and Preview

4.4.1 Badge Preview

Clicking the **Badge** button next to any employee navigates to the **Badge Preview** page (Figure 6.4). This page renders a pixel-accurate preview of the employee's identity card within the browser, including:

- The BadgeGen corporate header with logo and tagline "*Corporate Identity*".
- A circular employee photograph.
- The employee's full name in large bold font.
- Job title in a lighter weight below the name.
- Employee ID and department in a two-column metadata section.
- A QR code at the bottom encoding the employee ID.
- A blue gradient footer bar.

4.4.2 PDF Download

A **Download PDF** button in the top-right corner of the badge preview page triggers server-side PDF generation and delivers the badge as a downloadable file. This file is print-ready and formatted to standard ID card dimensions.

4.5 Navigation and Sidebar

The persistent left sidebar provides:

- The BadgeGen logo and tagline.
- Navigation links: **Dashboard**, **Employees**, **Bulk Upload**.
- The currently logged-in username at the bottom.
- A **Logout** button styled with a red border.

Chapter 5

Implementation Highlights

5.1 Employee Model

The central data model is the `Employee` Django model. A representative definition is shown in Listing 5.1.

```
import uuid
from django.db import models

DEPARTMENT_CHOICES = [
    ('Engineering', 'Engineering'),
    ('HR', 'Human Resources'),
    ('Finance', 'Finance'),
    ('Marketing', 'Marketing'),
]

GENDER_CHOICES = [
    ('Male', 'Male'),
    ('Female', 'Female'),
]

class Employee(models.Model):
    full_name     = models.CharField(max_length=200)
    job_title     = models.CharField(max_length=200)
    department    = models.CharField(max_length=100,
                                     choices=DEPARTMENT_CHOICES)
    gender        = models.CharField(max_length=10,
                                     choices=GENDER_CHOICES)
    photo         = models.ImageField(upload_to='employees/')
    employee_id   = models.CharField(max_length=20,
                                     unique=True, editable=False
                                        )
    created_at    = models.DateTimeField(auto_now_add=True)

```

```python
    def save(self, *args, **kwargs):
        if not self.employee_id:
            hex_part = uuid.uuid4().hex[:4].upper()
            year = self.created_at.year if self.created_at \
                   else 2026
            self.employee_id = f"EMP-{year}-{hex_part}"
        super().save(*args, **kwargs)

    def __str__(self):
        return f"{self.full_name} ({self.employee_id})"
```

Listing 5.1: Employee model (representative)

5.2 Badge PDF Generation View

The badge generation view retrieves the employee record, renders the badge layout, and returns the result as a PDF binary response:

```python
from django.http import HttpResponse
from django.shortcuts import get_object_or_404
import qrcode, io

def download_badge_pdf(request, employee_id):
    employee = get_object_or_404(Employee,
                                 employee_id=employee_id)
    # Generate QR code
    qr = qrcode.make(employee.employee_id)
    qr_buffer = io.BytesIO()
    qr.save(qr_buffer, format='PNG')
    qr_buffer.seek(0)

    # Render badge to PDF (using chosen PDF library)
    pdf_bytes = render_badge_to_pdf(employee, qr_buffer)

    response = HttpResponse(pdf_bytes,
                            content_type='application/pdf')
    response['Content-Disposition'] = (
        f'attachment; filename="badge_{employee.employee_id}.
           pdf"'
    )
    return response
```

Listing 5.2: Badge PDF generation view (representative)

5.3 Deployment Script

The project ships with a PowerShell script (`run-app.ps1`) that automates virtual environment activation and Django server startup, making local deployment a single command:

```
# run-app.ps1
Set-Location $PSScriptRoot
.\.venv\Scripts\Activate.ps1
python manage.py runserver
```

Listing 5.3: PowerShell launch script

Chapter 6

Application Screenshots

This chapter presents the key screens of the BadgeAppFTP application as captured during a live demonstration session.

6.1 Login Page

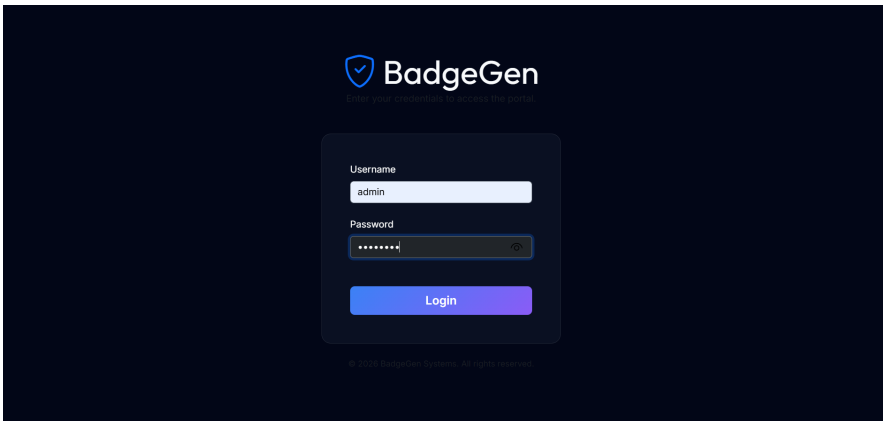


Figure 6.1: Login page – Administrator authentication interface

The login screen (Figure 6.1) greets the administrator with the BadgeGen logo and a clean dark-themed credential form. Only authenticated users can proceed to the management interface.

6.2 Dashboard

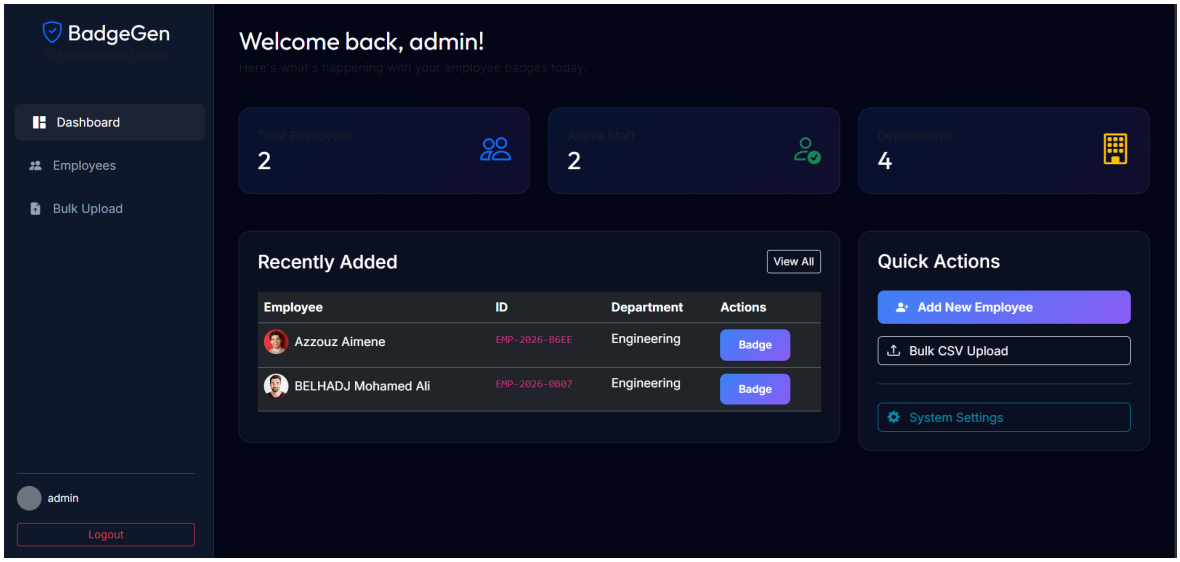


Figure 6.2: Main dashboard – System overview with metric cards and recent employees

The dashboard (Figure 6.2) provides a consolidated view of the organization's badge status. The three metric cards at the top display real-time counts, while the *Recently Added* table lists the most recent employee entries.

6.3 Add New Employee Form

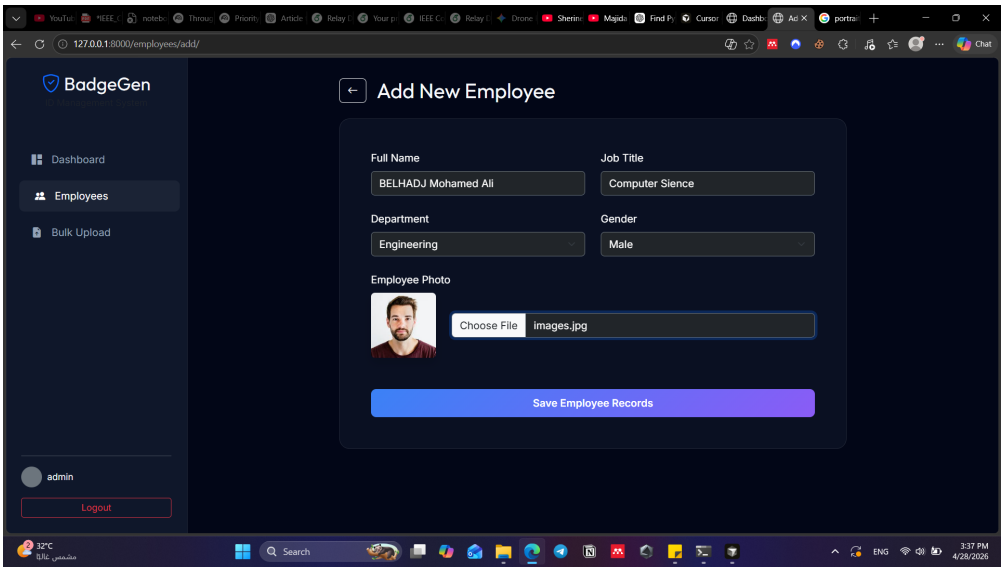


Figure 6.3: Add New Employee form – Data entry with photo upload

Figure 6.3 shows the employee registration form with all input fields populated. The uploaded photo is previewed inline before submission, allowing the administrator to

verify the image before saving the record.

6.4 Badge Preview and PDF Export

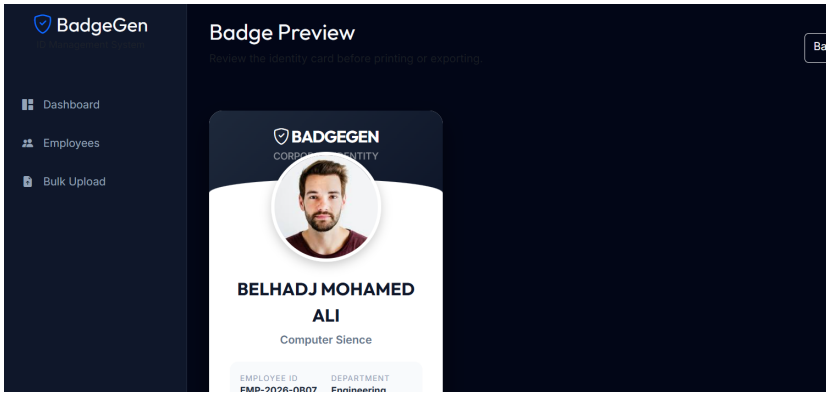
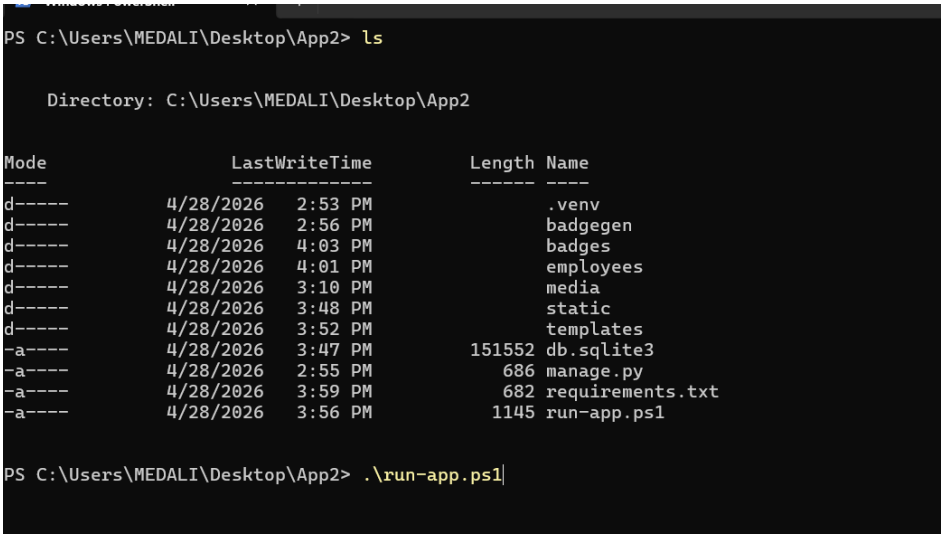


Figure 6.4: Badge Preview page – Rendered identity card for BELHADJ Mohamed Ali

Figure 6.4 demonstrates the generated identity card for employee `EMP-2026-0B07`. The badge displays the corporate header, circular photo, full name, job title, department, employee ID, and a QR code. The **Download PDF** button in the top-right corner exports the badge as a print-ready file.

6.5 Project File Structure (Terminal)



```
PS C:\Users\MEDALI\Desktop\App2> ls


    Directory: C:\Users\MEDALI\Desktop\App2


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         4/28/2026   2:53 PM                .venv
d-----         4/28/2026   2:56 PM                badgegen
d-----         4/28/2026   4:03 PM                badges
d-----         4/28/2026   4:01 PM                employees
d-----         4/28/2026   3:10 PM                media
d-----         4/28/2026   3:48 PM                static
d-----         4/28/2026   3:52 PM                templates
-a----         4/28/2026   3:47 PM         151552 db.sqlite3
-a----         4/28/2026   2:55 PM            686 manage.py
-a----         4/28/2026   3:59 PM            682 requirements.txt
-a----         4/28/2026   3:56 PM           1145 run-app.ps1


PS C:\Users\MEDALI\Desktop\App2> .\run-app.ps1
```

Figure 6.5: PowerShell terminal showing project directory structure and launch command

Figure 6.5 shows the project directory as seen in a Windows PowerShell terminal, confirming the presence of all key directories and files: the virtual environment, application modules, database file, and the `run-app.ps1` launch script.

Chapter 7

Testing and Validation

7.1 Manual Testing

The application was validated through a series of manual test scenarios covering all major user workflows:

1. **Authentication:** Verified that invalid credentials are rejected and valid ones grant access.
2. **Employee Creation:** Tested form submission with various inputs including boundary values for field lengths.
3. **Photo Upload:** Verified JPEG and PNG formats are accepted and previewed correctly.
4. **ID Uniqueness:** Confirmed that no two employees receive the same `EMP-YYYY-XXXX` identifier.
5. **Badge Generation:** Validated that the preview accurately reflects stored data and the downloaded PDF matches the preview.
6. **QR Code:** Scanned generated QR codes to confirm they encode the correct employee ID.
7. **Bulk Upload:** Tested CSV import with valid and malformed files to confirm error handling.

7.2 Observed Results

During the demonstration session:

- Two employee records were successfully created: *Azzouz Aimene* (`EMP-2026-B6EE`) and *BELHADJ Mohamed Ali* (`EMP-2026-0B07`).
- Both employees were assigned to the **Engineering** department.
- Badge preview and PDF download functioned correctly for both records.

- The dashboard correctly displayed **2 Total Employees**, **2 Active Staff**, and **4 Departments**.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

BadgeAppFTP successfully demonstrates a complete end-to-end employee badge management system built on modern web technologies. The project achieves its primary objectives:

- Providing a secure, authenticated administrative interface.
- Automating unique employee ID generation.
- Enabling high-quality badge generation with QR code support.
- Offering a professional PDF export ready for printing.
- Supporting bulk data import via CSV for large-scale use.

The choice of Django as the backend framework, combined with SQLite for persistence and a server-side PDF rendering pipeline, results in a self-contained, easily deployable application suitable for institutional use.

8.2 Future Work

Several enhancements are planned or proposed for future iterations:

1. **Production Deployment:** Migration from SQLite to PostgreSQL and deployment on a production server (e.g., Gunicorn + Nginx).
2. **Role-Based Access Control:** Introduction of HR Manager and Department Admin roles with scoped permissions.
3. **Multiple Badge Templates:** Allow administrators to choose from different visual badge layouts and color themes.
4. **REST API:** Expose employee and badge data via a RESTful API to support mobile applications and third-party integrations.

5. **FTP/Email Distribution:** Automated delivery of generated badges to employees via FTP server upload or email attachment (aligning with the "FTP" component of the project name).
6. **Employee Self-Service Portal:** A read-only portal allowing employees to view and download their own badge.
7. **Advanced Search and Filtering:** Full-text search and multi-criteria filtering in the employee list view.
8. **Audit Logs:** Recording of all administrative actions for compliance and traceability.

Bibliography

[1] Django Software Foundation. (2024). *Django Documentation – Version 5.x.*
https://docs.djangoproject.com/

[2] Python Software Foundation. (2024). *Python 3 Documentation.*
https://docs.python.org/3/

[3] Clark, A. et al. (2024). *Pillow – The Friendly PIL Fork.*
https://pillow.readthedocs.io/

[4] Lincoln, L. (2023). *qrcode – QR Code image generator.*
https://pypi.org/project/qrcode/

[5] ReportLab Inc. (2024). *ReportLab PDF Library – User Guide.*
https://www.reportlab.com/docs/reportlab-userguide.pdf

[6] Hipp, D. R. (2024). *SQLite Documentation.*
https://www.sqlite.org/docs.html

[7] BELHADJ Mohamed Ali. (2026). *BadgeAppFTP – Source Code Repository.* GitHub.
https://github.com/blhjmedali/BadgeAppFTP